

An Automatic Generator of Non-Power-of-Two SDF FFT Architectures for 5G and Beyond

Víctor Manuel Bautista

Department of Electronic Engineering
Universidad Politécnica de Madrid
Madrid, Spain
victor.bautista@upm.es

Mario Garrido

Department of Electronic Engineering
Universidad Politécnica de Madrid
Madrid, Spain
mario.garrido@upm.es

Abstract—This paper presents an automatic generator for non-power-of-two (NP2) single-path delay feedback (SDF) fast Fourier transform (FFT) architectures. Previous generators support sizes that are powers of two. Conversely, the sizes that are considered in the proposed generator include products of powers of 2, 3, and 5, which are the sizes reported in the physical layer description of 5G. The proposed approach not only includes the software tool but also utilizes hardware techniques to optimize the architectures. To measure the capabilities of the proposed generator, FFTs of a wide range of sizes have been implemented and reported, achieving high-performance capabilities.

Index Terms—Fast Fourier transform (FFT), non-power-of-two (NP2) size, single-path delay feedback (SDF), generator, VHDL.

I. INTRODUCTION

The fast Fourier transform (FFT) is an essential component of the physical layer of 5G [1]. The FFT algorithm allows to transform a sequence from the time domain to the frequency domain. 5G systems require the design of high-performance FFTs in hardware. In this context, we can distinguish between memory-based [2] and pipelined FFTs [3]. Memory-based FFTs use processing elements and memories to calculate the FFT iteratively. Pipelined FFT architectures process a continuous data flow. Depending on the area, latency, and power requirements, pipeline FFTs can have various degrees of parallelization. We can distinguish among parallel [4], [5], fully parallel [6], or serial architectures [7], [8].

The physical layer description of 5G considers FFT sizes that can be calculated as a product of powers of 2, 3, and 5 [1]. Thus, new designs that involve non-power-of-two (NP2) sizes are needed. The most popular architecture for processing NP2 FFTs is the single-path delay feedback (SDF) architecture [7], [8]. This is a type of serial pipelined architecture, so it processes one sample per clock cycle.

Previous works presented generators to implement pipelined FFTs according to the size and parallelization required [4]–[9]. [4] is provided as a highly parameterizable VHDL code

This work was supported in part by MCIN/AEI/10.13039/501100011033 and "ERDF A way of making Europe" under Project PID2021-126991NA-I00; and in part by MCIN/AEI/10.13039/501100011033 and "ESF Investing in your future" under Grant RYC2018-025384-I.

where the user can set the size and the parallelization of the FFTs. [9] is a web generator that provides the Verilog code of the required FFT. [5] generates the Verilog code of multiple-path delay feedback architectures (MDF). [6] generates the VHDL code of fully parallel architectures. Finally, [7] and [8] generate SDF architectures in VHDL and Verilog, respectively. All these previous works are restricted to power-of-two sizes of the FFT and do not consider radix-3 and radix-5 algorithms.

This paper presents a new generator of SDF FFT architectures that supports the sizes considered in 5G and future communication systems. The software tool not only provides FFTs of a wide range of sizes but also utilizes hardware optimization techniques to optimize the architectures. As a result, the VHDL code that is generated obtains high-performance SDF FFTs and supports non-power-of-two sizes.

The paper is organized as follows: In Section II, the state-of-the-art is reviewed. In Section III, the proposed software tool is presented. In Section IV, an example of a generated architecture is provided and implementation considerations are discussed. In Section V, experimental results for various FFT sizes FFT are reported. In Section VI, the proposed generator is compared with previous ones. Finally, in Section VII, the main conclusions of this paper are extracted.

II. BACKGROUND

A. The FFT

The N -point discrete Fourier transform (DFT) of an input sequence $x[n]$ is calculated as

$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot W_N^{nk}, \quad k = 0, 1, \dots, N-1, \quad (1)$$

where $X[k]$ represents the output value at the frequency k . The factor $W_N^{nk} = e^{-\frac{2\pi}{N}nk}$ is the twiddle factor and carries out multiplications in the complex plane. The FFT algorithm reduces the number of operations from $\mathcal{O}(N^2)$ in the DFT to $\mathcal{O}(N \log N)$. The FFT algorithm divides the N -point FFT into smaller FFTs whose product is equal to N .

B. Cooley-Tukey Algorithm for NP2 Sizes

Fig. 1 shows the flow graph of a 6-point FFT according to the Cooley-Tukey algorithm [10]. The flow graph is divided

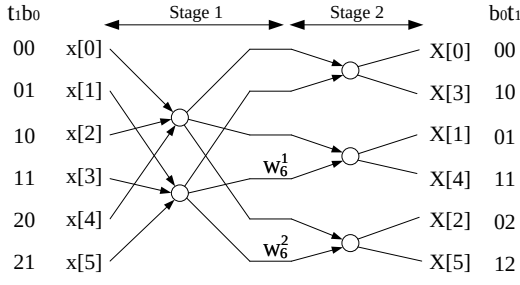


Fig. 1. Flow graph of a 6-point FFT.

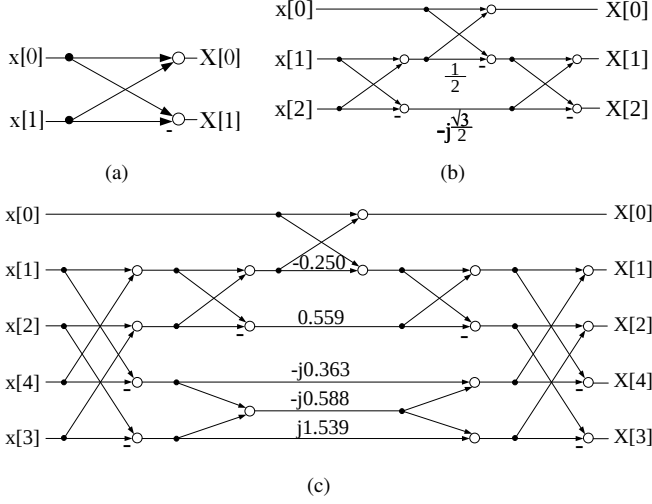


Fig. 2. FFT butterflies. (a) Radix-2. (b) Radix-3. (c) Radix-5.

into two stages. The first stage includes two radix-3 butterflies, which are represented by a circle, and the second stage consists of three radix-2 butterflies. A radix- r butterfly calculates an r -point FFT. Between both sets of butterflies, two multiplications by W_6^1 and W_6^2 have to be calculated. Note that the input data are provided in natural order when reading them from top to bottom in the flow graph. By contrast, the output data are provided in bit-reversed order.

The binary representation of the input and output indexes considers the binary bit b_0 and the ternary bit t_1 . Input data to be operated in the same butterfly at stage 1 share the same binary representation except for the most significant bit, t_1 . This way, $x[0]$, $x[2]$ and $x[4]$ are operated in the upper radix-3 butterfly, and $x[1]$, $x[3]$ and $x[5]$ are operated in the lower radix-3 butterfly. In the second stage, the calculations are divided into three 2-point FFTs. In this case, data coming into a butterfly differ in b_0 .

C. Butterflies in 5G

A radix- r butterfly calculates an r -point FFT. Fig. 2(a) shows the flow graph of a radix-2 butterfly, which is the particularization of (1) for $N = 2$. It consists of one complex addition and one complex subtraction. As an adder and a subtractor have similar complexity, throughout this paper we refer to both of them as adders. Thus, the implementation of the radix-2 butterflies requires four real adders.

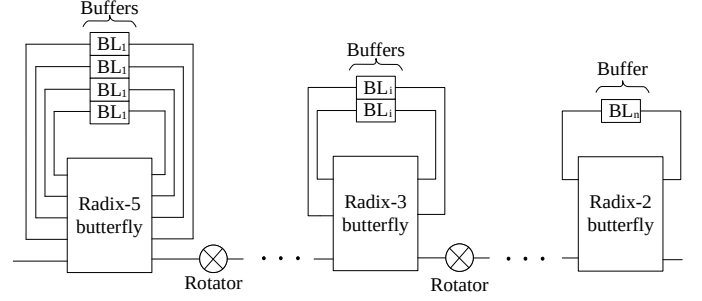


Fig. 3. Example of a generic SDF FFT architecture.

Fig. 2(b) shows the flow graph of the radix-3 butterfly derived from the Rader's algorithm [11]. The multiplication by $1/2$ is a bit shift that is implemented in hardware at no cost. The multiplication by the pure imaginary constant $j\sqrt{3}/2$ can be calculated with 2 real multiplications. As a result, the radix-3 butterfly requires 12 real adders and two real multipliers.

Fig. 2(c) shows the flow graph of the radix-5 butterfly proposed in [12], which differs slightly from the radix-5 butterfly proposed by Winograd [13]. In [12], the operations are reordered to make the top multiplication a power of two, which is implemented with a bit-shift. Contrarily to the top multiplication, the other multiplications are not by powers of two. This fact entails the implementation of two real multipliers for each value. As a result, the radix-5 butterfly in Fig. 2(c) requires 34 real adders and 8 real multipliers.

D. SDF FFT Architecture for 5G

Fig. 3 shows a generic N -point SDF FFT architecture that considers the FFT sizes supported in 5G, including radix-2, radix-3, and radix-5 butterflies. As SDF FFTs are serial architectures, only one input and one output path are used.

The SDF architecture mainly consists of butterflies, rotators, and buffers. The rotators carry out multiplications by the twiddle factors. Each butterfly has $r - 1$ branches connected to buffers. The product of all the radices of the butterflies in the architecture is equal to N .

The sizes of the buffers at stage i (BL_i) are determined by the separation in clock cycles between inputs that must be operated together in the corresponding butterfly. To clarify this point, let us consider the example of the 6-point FFT in Fig. 1. In an SDF architecture, inputs arrive from top to bottom in the flow graph, at one input per clock cycle. Once the processing starts, the first data have to be stored in the buffers until the last input of the same butterfly arrives. Consequently, the SDF architecture of the 6-point FFT of Fig. 1 would require buffers of sizes 2 and 1 at stages 1 and 2, respectively.

III. PROPOSED SOFTWARE TOOL

This paper proposes a software tool that automatically generates the VHDL code of SDF FFT architectures for NP2 sizes. It has been developed in Python and uses several parameterizable VHDL files. Table I shows the configuration parameters of the tool and their default values. The generator supports any FFT size required in 5G, i.e. any product of

TABLE I

CONFIGURATION PARAMETERS FOR THE PROPOSED SOFTWARE TOOL.

Parameter	Configuration	Default
FFT Length (N)	Any $2^{\alpha_2} \cdot 3^{\alpha_3} \cdot 5^{\alpha_5}$	1200 points
Radix Order	Any	5, 3, 2
Data Word Length (WL)	Any integer < 18	16 bits
Coeff. Word Length (WLC)	Any integer < 18	16 bits
Number Representation	Integer values (fixed)	-
Input Data Order	Natural (fixed)	-
Output Data Order	Bit-reversed (fixed)	-
Use Pipeline (PIPE)	0 or 1	1
No. of FFTs to test (NFFT)	Any integer	1

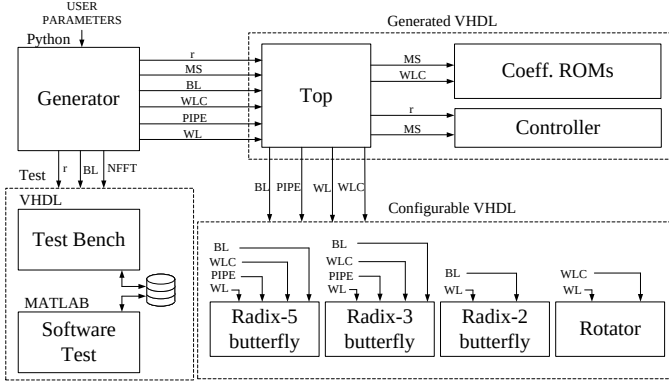


Fig. 4. File distribution of the project.

powers of 2, 3, and 5. Furthermore, it supports any order of the radices along FFT stages. The word length for data and coefficients can be any integer lower than 18 bits specified by the user. Their default value is 16 bits. No increase in the data word length along the stages is considered. The generated circuit has natural input data order and bit-reversed output data order. The PIPE parameter allows to use pipeline registers (PIPE = 1) inside the radix-3 and radix-5 butterflies to reach high clock frequencies. The last parameter, NFFT, sets the number of FFTs processed in the test bench.

Fig. 4 shows the files used in the tool. The main file is the Python file called generator. This is the file that the user has to execute. The user can set the parameters according to Table I. The main file generates the code of the VHDL top file, the controller, and $n - 1$ coefficient read-only memory (ROM) files, where n is the number of stages. The VHDL code of the butterflies and the rotators are provided as parameterizable VHDL code. Their parameters are configured in the top file. The generator also generates the VHDL file needed to test the architecture. This file communicates with MATLAB software through a database, which is also provided in the project.

By executing the generator, the program first extracts the memory sizes, the length of the buffers, and radices. The radices are defined as

$$r = \{r_1 \ r_2 \ \dots \ r_n\}, \quad (2)$$

for stages $1, \dots, n$. The product of all the radices is N .

The set of ROM sizes can be described as

$$MS = \{MS_1 \ MS_2 \ \dots \ MS_{n-1}\}. \quad (3)$$

Each size MS_i is calculated as the product of the radices in the current and next stages, i.e.,

$$MS_i = \prod_{j=i}^n r_j. \quad (4)$$

To minimize the total memory size in the architecture, by default, the generator places radix-5 butterflies first, followed by radix-3 butterflies, and finally radix-2 butterflies. However, the user can specify any order for the radices, as shown in Table I. As a result, the tool generates $n - 1$ different memory files with the values of the twiddle factors.

The set of buffer lengths can be written as

$$BL = \{BL_1 \ BL_2 \ \dots \ BL_n\}. \quad (5)$$

Each length BL_i is calculated as

$$BL_i = \prod_{j=i+1}^n r_j. \quad (6)$$

Each stage has a butterfly of radix- r_i connected to $r_i - 1$ buffers of size BL_i . The sum of the length of all these buffers is equal to $N - 1$, despite the radices involved.

The sets (2), (3) and (5) are used as parameters to instantiate the components in a loop of n iterations. In a general case, the top file instantiates n butterflies according to (2) and $n - 1$ memories according to (3), connected to $n - 1$ rotators. The buffers are integrated inside the butterflies and are configured according to (5). Their lengths are parameterizable and are configured in the top file when the butterfly is instantiated. The control module is integrated into the top file.

IV. IMPLEMENTATION

Fig. 5 shows the implementation of a 1200-point SDF FFT architecture automatically designed by the proposed generator. The generated architecture has two radix-5 butterflies, one radix-3 butterfly, and four radix-2 butterflies. This leads to

$$\begin{aligned} r &= \{5 \ 5 \ 3 \ 2 \ 2 \ 2 \ 2\}, \\ BL &= \{240 \ 48 \ 16 \ 8 \ 4 \ 2 \ 1\}, \\ MS &= \{1200 \ 240 \ 48 \ 16 \ 8 \ 4\}, \end{aligned} \quad (7)$$

which corresponds to the values in the figure. For the rotators after the butterflies, we have used the designs in [14]. They use three DSPs to carry out the rotations.

For any N , the latency of a generated architecture in clock cycles is calculated as

$$\text{Lat} = N - 1 + (n - 1) \cdot \text{Lat}_{rot} + n + \sum_{i=0}^{n-1} \text{Lat}(r_i). \quad (8)$$

The term $N - 1$ is the sum of all the buffers in BL . Lat_{rot} is the latency of each of the $n - 1$ rotators, which is equal to six clock cycles. The term n corresponds to the output register of each butterfly. The last term represents the latency due to the pipeline of the butterflies, which is equal to zero

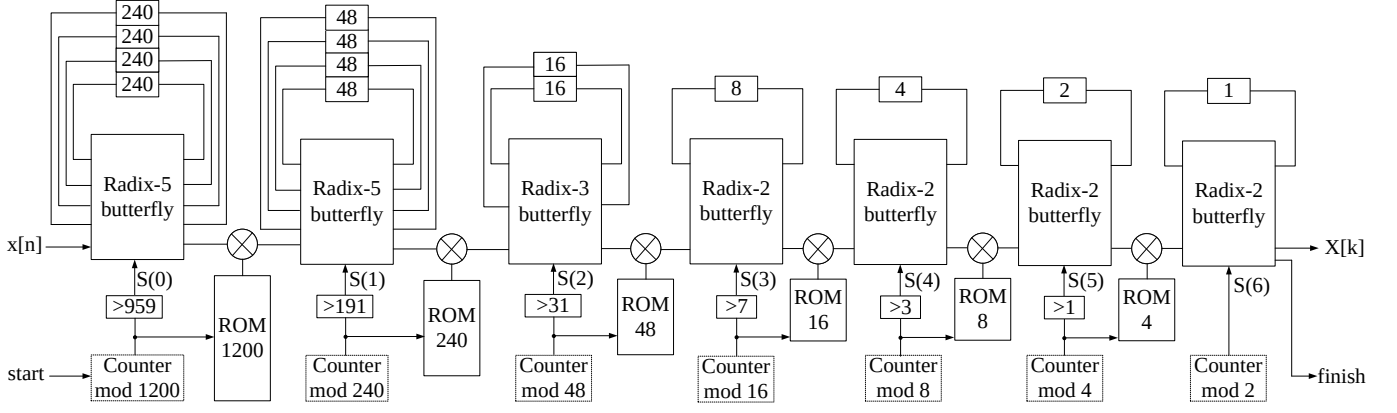


Fig. 5. Implementation of a 1200-point SDF FFT architecture generated by the proposed software tool.

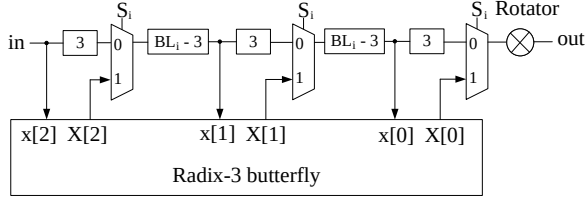


Fig. 6. Proposed radix-3 SDF stage with 3 clock cycles of pipeline in the butterfly.

for radix-2, three for radix-3, and five for radix-5 butterflies. It can be observed that the latency of the 1200-point FFT shown in Fig. 5 is 1255 clock cycles.

A. Pipelining Technique in SDF Butterflies

To generate high-throughput architectures, pipeline registers have been added in the radix-3 and radix-5 butterflies to increase the clock frequency. Fig. 6 shows a radix-3 SDF stage according to the representation in [3], but using the pipelining technique. Originally, the buffers have length BL_i . These buffers are split into a buffer of length $BL_i - 3$ and a buffer of length 3. The buffer with length 3 is propagated back through the multiplexers. This places a buffer of length 3 at the upper input of the multiplexers and, at the same time provides 3 registers to each output of the radix-3 butterfly. These registers are used to add pipelining to the butterfly and increase the clock frequency.

B. Control Circuit

The control circuit handles the control signals of the multiplexers and the address signals of the ROMs. Each stage of Fig. 5 has one counter that operates modulo MS_i and one comparator. The arrival of new data at the input of each stage resets its counter. The address signals for the ROMs are directly connected to the counters. To adapt to the delay of the data through the stage, the coefficients stored in the ROMs are shifted circularly. Thus, the first coefficient is stored at the ROM address that is equal to the latency of the stage. This avoids adding delays between the counters and the ROMs.

The counters provide the control signals for the multiplexers. As butterflies only need to process data the last $1/r_i$ of the

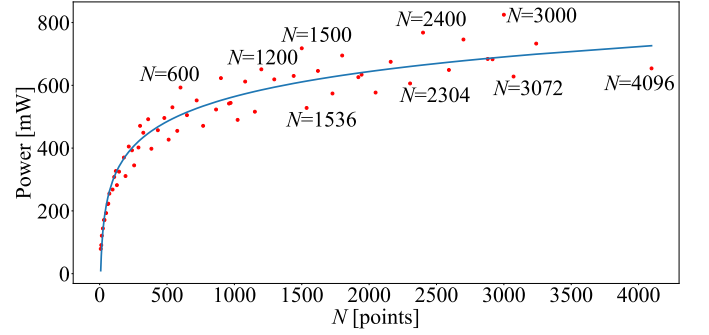


Fig. 7. Power consumption vs. N for the generated FFT architectures.

time, each signal S_i is set to one only after the corresponding counter reaches the value $MS_i - MS_i/r_i$, which is determined with a comparator.

V. EXPERIMENTAL RESULTS

To show the capabilities of the proposed tool, 63 different FFT sizes have been generated and implemented on a Virtex Ultraescale+ XCVU37P-FSVH2892-2L-E. Table II shows the post-implementation results. The area is measured in terms of CLB LUTs, CLB REGs, CARRY8s, CLBs, BRAMs, and DSP Slices (DSPs). The table also includes the factors of each N , clock frequency (f_{CLK}), signal-to-quantization-noise ratio (SQNR) calculated according to [15], latency, and power consumption. The real and imaginary parts of data and coefficients have each a word length of 16 bits. It can be observed that most of the designs work at a high frequency of 500 MHz thanks to the pipelining that has been applied to the architectures.

The SQNR has a decreasing tendency with N and is affected by the radices. Radix-5 stages have the highest contribution to the quantization noise, followed by radix-3 and radix-2 ones.

The latency is obtained from (8). It can be observed that it follows an approximately linear pattern with respect to N .

The power consumption of the architectures is measured at 500 MHz and is shown graphically in Fig. 7. The dots show the exact values and the curve is the logarithmic regression of these values, which corresponds to

$$\text{Power (mW)} \approx -229.05 + 114.84 \cdot \log_{10} N. \quad (9)$$

TABLE II
POST-IMPLEMENTATION RESULTS FOR DIFFERENT SIZES OF FFTS USING THE PROPOSED TOOL ON A VIRTEX XCVU37P-FSVH2892-2L-E.

N	FFT size Factors	Area						f_{CLK} (MHz)	SQNR (dB)	Latency (cycles)	Power (mW)
		CLB LUT	CLB REG	CARRY8	CLB	BRAM	DSP				
8	{2 2 2}	456	476	36	106	0	6	500	77.76	22	79
12	{3 2 2}	611	867	60	156	0	8	500	72.54	26	91
16	{2 2 2 2}	668	662	48	163	0	9	500	73.66	37	121
24	{3 2 2 2}	863	1158	72	218	0	11	500	69.04	45	144
32	{2 2 2 2 2}	895	868	60	205	0	12	500	70.03	60	171
36	{3 3 2 2}	1036	1563	96	276	0	13	500	64.95	57	171
48	{3 2 2 2 2}	1091	1361	84	276	0	14	500	65.64	76	193
60	{5 3 2 2}	1759	2106	162	408	0	19	500	60.95	81	222
64	{2 2 2 2 2 2}	1117	1073	72	274	0	15	500	66.80	99	223
72	{3 3 2 2 2}	1193	1858	108	337	0.5	16	500	61.49	100	255
96	{3 2 2 2 2 2}	1221	1567	96	296	0.5	17	500	62.43	131	268
108	{3 3 3 2 2}	1372	2233	132	375	1	18	500	57.36	136	308
120	{5 3 2 2 2}	1851	2434	174	463	0.5	22	500	57.62	148	327
128	{2 2 2 2 2 2 2}	1368	1281	84	295	0	18	500	63.63	170	282
144	{3 3 2 2 2 2}	1424	2032	120	381	1	19	500	58.22	179	325
180	{5 3 3 2 2}	1999	2809	198	493	1	24	500	53.50	208	370
192	{3 2 2 2 2 2 2}	1482	1774	108	365	0.5	20	500	59.35	234	311
216	{3 3 3 2 2 2}	1503	2514	144	406	2	21	500	54.13	251	405
240	{5 3 2 2 2 2}	2113	2608	186	488	1	25	500	54.37	275	393
256	{2 2 2 2 2 2 2 2}	1722	1491	96	361	0	21	500	60.56	305	345
288	{3 3 2 2 2 2 2}	1559	2258	132	411	2	22	500	55.09	330	402
300	{5 5 3 2 2}	2784	3355	264	619	1	30	500	49.73	328	471
324	{3 3 3 3 2 2}	1681	2809	168	425	2.5	23	500	50.00	359	449
360	{5 3 3 2 2 2}	2138	2916	210	533	3	27	500	50.33	395	492
384	{3 2 2 2 2 2 2 2}	1670	1971	120	414	1.5	23	500	56.24	433	398
432	{3 3 3 2 2 2 2}	1724	2609	156	438	2.5	24	500	50.90	474	457
480	{5 3 2 2 2 2 2}	2166	2591	198	504	3	28	500	51.23	522	496
512	{2 2 2 2 2 2 2 2 2}	2125	1639	108	460	0.5	24	500	57.49	568	427
540	{5 3 3 3 2 2}	2310	3258	234	583	4	29	500	46.24	575	530
576	{3 3 2 2 2 2 2 2}	1787	2467	144	470	2.5	25	500	52.02	625	455
600	{5 5 3 2 2 2}	2784	3461	276	661	3.5	33	500	46.49	635	593
648	{3 3 3 3 2 2 2}	1807	3011	180	458	4	26	500	46.83	690	505
720	{5 3 3 2 2 2 2}	2356	3058	222	578	4	30	500	47.11	762	552
768	{3 2 2 2 2 2 2 2 2}	2052	2069	132	465	2	26	500	53.22	824	471
864	{3 3 3 2 2 2 2 2}	1883	2839	168	487	4	27	500	47.80	913	523
900	{5 5 3 3 2 2}	2928	3837	300	673	4	35	500	42.40	935	623
960	{5 3 2 2 2 2 2 2}	2415	2800	210	574	3.5	31	500	48.20	1009	542
972	{3 3 3 3 3 2 2}	1998	3388	204	531	4.5	28	500	42.74	1014	544
1024	{2 2 2 2 2 2 2 2 2 2}	2305	1821	120	533	2	27	500	54.43	1087	490
1080	{5 3 3 3 2 2 2}	2441	3460	246	574	6	32	500	42.98	1122	612
1152	{3 3 2 2 2 2 2 2 2}	1981	2436	156	510	4.5	28	500	48.95	1208	516
1200	{5 5 3 2 2 2 2}	3050	3637	288	690	5	36	500	43.27	1255	651
1200	{2 2 2 2 3 5 5}	3227	1541	288	651	5.5	36	166	45.42	1242	200*
1200	{2 5 3 5 2 2 2}	2978	3736	288	732	7	36	500	43.41	1255	661
1200	{2 2 5 5 3 2 2}	3173	3595	288	725	5.5	36	500	43.64	1255	635
1296	{3 3 3 3 2 2 2 2}	3369	3316	192	830	3.5	29	500	43.63	1345	619
1440	{5 3 3 2 2 2 2 2}	2507	3334	234	671	6	33	500	43.97	1489	630
1500	{5 5 5 3 2 2}	3724	4385	366	853	5	41	500	38.62	1535	718
1536	{3 2 2 2 2 2 2 2 2 2}	2462	2219	144	527	3.5	29	500	50.18	1599	528
1620	{5 3 3 3 3 2 2}	2618	3837	270	649	6.5	34	500	38.93	1662	646
1728	{3 3 3 2 2 2 2 2 2 2}	2102	2933	180	539	6.5	30	500	44.72	1784	574
1800	{5 5 3 3 2 2 2}	3063	3912	312	724	7	38	500	39.21	1842	695
1920	{5 3 2 2 2 2 2 2 2}	2628	3015	222	661	5.5	34	500	45.15	1976	626
1944	{3 3 3 3 3 2 2 2}	2195	3756	216	592	8	31	500	39.53	1993	634
2048	{2 2 2 2 2 2 2 2 2 2 2}	2501	2072	132	589	5	30	500	51.43	2118	577
2160	{5 3 3 3 2 2 2 2}	2674	3638	258	657	8.5	35	500	39.80	2209	675
2304	{3 3 2 2 2 2 2 2 2 2}	2361	2873	168	616	8	31	500	45.92	2367	606
2400	{5 5 3 2 2 2 2 2}	5017	3672	300	1093	5	39	500	40.18	2449	768
2592	{3 3 3 3 2 2 2 2 2 2}	2213	3497	204	595	10	32	500	40.50	2648	649
2700	{5 5 3 3 3 2 2}	3258	4289	336	763	12	40	500	35.15	2742	746
2880	{5 3 3 2 2 2 2 2 2 2}	2731	3448	246	690	10.5	36	500	40.90	2936	684
2916	{3 3 3 3 3 3 2 2}	2305	3970	240	639	10.5	33	500	35.46	2965	683
3000	{5 5 5 3 2 2 2}	3749	4492	378	883	11.5	44	500	35.38	3042	825
3072	{3 2 2 2 2 2 2 2 2 2 2}	2656	2522	156	651	8	32	500	47.16	3142	628
3240	{5 3 3 3 3 2 2 2}	2753	4171	282	739	12	37	500	35.72	3289	733
4096	{2 2 2 2 2 2 2 2 2 2 2 2}	2700	2196	144	606	11	33	500	48.40	4173	654

*: Power consumption measured at 166 MHz

TABLE III
COMPARISON AMONG FFT GENERATORS

Parameter	[4]	[9]	[5]	[8]	[6]	[7]	Proposed
Programming Language	VHDL	Scala	C++	Scala	Matlab/C++	-	Python/VHDL
Generated Language	VHDL	Verilog	Verilog	VHDL	VHDL	Verilog	VHDL
Architecture Type	MDC	Streamed	MDF	SDF	FP	SDF	SDF
FFT Size	16-65536	4-32768	Any 2^k	Any 2^k	8-256	8-65536	Any $2^{\alpha_2} 3^{\alpha_3} 5^{\alpha_5}$
Parallelization	4-64	2-64	1-2	1	= FFT Size	1-12	1
Radix	2 and 2^2	2-64	2	2, 4 and 2^2	2^k /SR	2, 4 and 2^2	2, 3 and 5
Data WL	Any	Any	Any	Any	Any	8-34	Any WL < 18
Incremental Data WL	Optional	NP	Yes	Optional	No	Optional	No
Coefficient WL	Any	NP	Any	Any	WL_E	8-34	Any WL < 18
Number Format	FX	FX/FL	FX	FX	FX	FX/FL	FX
Input Order	NAT	NAT/BR	NAT	NAT	NAT	NAT	NAT
Output Order	BR	NAT/BR	NAT/BR	NAT/BR	NAT/BR	NAT/BR	BR
Pipelining	Optional	Included	Included	Optional	Included	Included	Optional

The components that contribute the most to the power consumption are the radix-5 butterflies because they incorporate the largest amount of arithmetic operations in the architecture. We can see in Fig. 7 that, in general, the architectures that demand more power, i.e., the ones that are above the regression line, have at least one radix-5 butterfly.

To show the impact of the order of the radices, four different implementations have been generated for 1200 points. They are shown in grey in Table II. The first implementation corresponds to the architecture shown in Fig 5, which places radix-5 butterflies first, followed by radix-3 butterflies and, finally, radix-2 butterflies. This minimizes the total memory usage, as discussed in Section III. The second implementation places the smallest butterflies first, in inverse order with respect to the first implementation. As the length of the buffer in the last stage is 1, it can not be used to pipeline the radix-5 butterfly efficiently. This reduces the maximum clock frequency to 166 MHz. The remaining two implementations consider intermediate alternatives that provide a different trade-off among resources. In all of them, the last two FFT stages are reserved for radix-2. Thus, guarantees that the butterflies in stages with radices 3 and 5 are pipelined properly, due to the higher buffer lengths, which guarantees a clock frequency of 500 MHz.

VI. COMPARISON

Table III compares the proposed generator to previous FFT generators in the literature. Most generators are developed in object-oriented languages, such as Python, Scala, or C++, except [4]. A wide variety of architectures is offered in the generators of the literature, including fully parallel (FP) [6], MDC [4], MDF [5] and SDF [7], [8] FFTs.

Regarding FFT sizes, it can be observed that the proposed generator is the only one that supports non-power-of-two sizes, which is the main contribution of this work.

As in all other generators, the proposed one supports fixed point (FX) data. Some previous generators also support floating point (FL) [7], [9]. The proposed generator considers natural (NAT) input data order and bit-reversed (BR) output order, which is common in FFT architectures. Finally, the proposed generator has the option to use pipelining, as in [4] and [8], providing the high performance reported in Section V.

VII. CONCLUSIONS

This work has presented a new generator of SDF FFT architectures. It is the first generator that supports power-of-two FFT sizes, which makes it suitable for 5G systems and beyond. The generator is developed in Python and VHDL, and generates VHDL code. Experimental results for 63 different sizes have been obtained, providing high-performance SDF FFT architectures.

REFERENCES

- [1] 3GPP, "3GPP TS 38.211 - Physical channels and modulation V16.3.0," Sep. 2020, <https://www.3gpp.org/DynaReport/38-series.htm>.
- [2] Z. Kaya, M. Garrido, and J. Takala, "Memory-based FFT architecture with optimized number of multiplexers and memory usage," *IEEE Trans. Circuits Syst. II*, 2023, Early access.
- [3] M. Garrido, "A survey on pipelined FFT hardware architectures," *J. Signal Process. Syst.*, vol. 94, no. 11, pp. 1345–1364, Nov. 2022.
- [4] M. Garrido, M. Acevedo, A. Ehliar, and O. Gustafsson, "Challenging the limits of FFT performance on FPGAs," in *Proc. Int. Symp. Integrated Circuits*, Dec. 2014, pp. 172–175.
- [5] D. Gisselquist and M. Nolan, "An open source pipelined FFT generator," Oct. 2018, <https://zipcpu.com/dsp/2018/10/02/fft.html>.
- [6] M. Garrido, K. Möller, and M. Kumm, "World's fastest FFT architectures: Breaking the barrier of 100 GS/s," *IEEE Trans. Circuits Syst. I*, vol. 66, no. 4, pp. 1507–1516, Apr. 2019.
- [7] "Fast Fourier transform v9.1 LogiCORE IP product guide," May 2022, <https://docs.xilinx.com/r/en-US/pg109-xfft/Features?tocId=ND69xvDQn~o2W3chWmrYDA>.
- [8] V. M. Milovanović and M. L. Petrović, "A highly parametrizable Chisel HCL generator of single-path delay feedback FFT processors," in *Proc. IEEE Int. Conf. Microelectron.*, Sep. 2019, pp. 247–250.
- [9] F. Franchetti, T. M. Low, D. T. Popovici, R. M. Veras, D. G. Spampinato, J. R. Johnson, M. Püschel, J. C. Hoe, and J. M. F. Moura, "Spiral: Extreme performance portability," *Proc. IEEE*, vol. 106, no. 11, pp. 1935–1968, Nov. 2018.
- [10] J. W. Cooley and J. W. Tukey, "An algorithm for the machine calculation of complex Fourier series," *Math. Comput.*, vol. 19, no. 90, pp. 297–301, Apr. 1965.
- [11] C. M. Rader, "Discrete Fourier transform when the number of data samples is prime," *Proc. IEEE*, vol. 56, no. 6, pp. 1107–1108, Jun. 1968.
- [12] V. M. Bautista and M. Garrido, "Serial butterflies for non-power-of-two FFT architectures in 5G and beyond," 2023, Manuscript under review.
- [13] S. Winograd, "On computing the discrete Fourier transform," *Math. Comput.*, vol. 32, no. 141, pp. 175–199, Jan. 1978.
- [14] P. Paz and M. Garrido, "Efficient implementation of complex multipliers on FPGAs using DSP slices," *J. Signal Process. Syst.*, 2023, Early access.
- [15] D. Guinart, "Deterministic analysis of the accuracy in FFT hardware architectures," Master's Thesis, Dpt. of Electrical Engineering, Linköping University, Jun. 2012.